

Node.js – Short Notes for Interviews

Quick revision sheet (Freshers: Full Stack / Web / Software Development)

1 Basics & Architecture

- **Node.js** is a JavaScript **runtime environment** (not a language) built on Chrome's **V8 engine**. It runs JS outside the browser.
- V8 compiles JavaScript to **machine code**. Node adds extras: **fs**, **http**, OS access.
- **Why Node.js**: single-threaded, non-blocking I/O, fast, same language on frontend and backend, huge NPM ecosystem.
- **Event-driven architecture**: register a callback, continue working, callback runs when the task finishes.
- **Sync vs Async**: sync runs line by line and waits; async starts a task and moves on.
- **Blocking vs Non-blocking I/O**: blocking waits for the operation; non-blocking continues and gets notified later.
- Best for **I/O-heavy** apps (APIs, chat, streaming). Not ideal for **CPU-heavy** tasks.

Aspect	Node.js	PHP / Java (traditional)
Threading	Single-threaded	Thread per request
I/O	Non-blocking	Often blocking
Language	JavaScript everywhere	Different backend language

2 Event Loop

- Mechanism that lets Node do **non-blocking I/O** on a single thread. Slow tasks are handed to **libuv** / the OS.
- **Key parts**: Call Stack, Node APIs/libuv, Callback Queue (macrotasks), Microtask Queue, Event Loop.
- **Phases (in order)**: Timers → Pending callbacks → Poll → Check → Close callbacks.
 - Timers: **setTimeout**, **setInterval**
 - Poll: I/O callbacks (most work happens here)
 - Check: **setImmediate**
- Microtasks (**process.nextTick**, Promises) are cleared **completely** between phases. **nextTick** runs before Promise callbacks.
- **Execution order**: sync code → **nextTick** → Promises → **setTimeout** → **setImmediate**.
- Inside an I/O callback, **setImmediate** always runs before **setTimeout(fn,0)**.
- **Concurrency**: JS is single-threaded, but libuv has a **thread pool** (default 4) for fs, DNS, crypto. Network I/O is handled by the OS.
- **Queue overload**: the queue has no fixed size, it grows in memory. Result: higher latency, then memory issues/crash. OS also has a **TCP backlog**.
- **Solutions**: clustering, load balancing, rate limiting, horizontal scaling, avoid blocking the loop.

3 Modules

- Module = reusable block of code with its own scope.
- **Types**: Core (**fs**, **http**), Local (your files), Third-party (NPM).

Feature	CommonJS	ES Modules
Syntax	require / module.exports	import / export
Loading	Synchronous	Asynchronous
Default in Node	Yes	No (" type ": " module " or .mjs)
Imports	Can be anywhere	Static, top level

- **module.exports vs exports**: **exports** is only a reference to **module.exports**. **require()** returns **module.exports**. Reassigning **exports** breaks the link.
- **Module caching**: first **require** runs and caches the module in **require.cache**. Later requires return the cached copy, so code runs once and state is shared (Singleton behaviour).
- **Module wrapper function**: (**exports**, **require**, **module**, **__filename**, **__dirname**) gives each file a private scope.
- **Circular dependency**: A requires B and B requires A gives a partial export (may be **undefined**).
- **Resolution order**: core module → relative path (**./**) with extensions and **index.js** → **node_modules** upward.
- Missing module gives error **MODULE_NOT_FOUND**.

4 Core Built-in Modules

Module	Key points
<code>fs</code>	File operations. Sync (<code>readFileSync</code> , blocking) vs async (callback) vs <code>fs/promises</code> . Prefer async in servers.
<code>http</code>	Create server with <code>createServer((req,res)=>{})</code> ; Express is built on it.
<code>path</code>	OS-independent paths: <code>join</code> (combine), <code>resolve</code> (absolute), <code>basename</code> , <code>dirname</code> , <code>extname</code> , <code>parse</code> .
<code>os</code>	System info: <code>platform()</code> , <code>cpus()</code> , <code>totalmem()</code> , <code>freemem()</code> .
<code>events</code>	<code>EventEmitter</code> : <code>on</code> , <code>emit</code> , <code>once</code> , <code>off</code> . Listeners run synchronously in order. <code>error</code> event with no listener crashes the app. Default max 10 listeners.
<code>util</code>	<code>util.promisify()</code> converts callback functions to Promises.
<code>url</code>	URL class, <code>searchParams</code> ; <code>querystring</code> parses query strings.
<code>stream</code>	Process data in chunks (see Streams section).

5 Asynchronous Programming

- **Callback**: function passed to run later. **Error-first** convention: `(err, data)`.
- **Callback hell**: deeply nested callbacks, hard to read and debug.
- **Promise states**: **pending**, **fulfilled**, **rejected**. Once settled, it never changes. Use `.then()`, `.catch()`, `.finally()`. Chaining avoids nesting.
- **async/await**: syntactic sugar over Promises. `async` function always returns a Promise. `await` pauses only that function, not the event loop.
- **Error handling**: callbacks → check `err`; Promises → `.catch()`; `async/await` → `try/catch`.

Method	Behaviour
<code>Promise.all</code>	Waits for all; rejects if <i>any</i> rejects
<code>Promise.allSettled</code>	Waits for all; never rejects; gives each status
<code>Promise.race</code>	First to <i>settle</i> (success or failure)
<code>Promise.any</code>	First to <i>fulfill</i> ; rejects only if all reject

- **Parallel** execution: `Promise.all`. **Sequential** only when tasks depend on each other.
- Promise callbacks are **microtasks**, timers/I/O are **macrotasks**. Microtasks run first.
- Unhandled rejection crashes the process in Node 15+.

```
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
console.log('sync');
// Output: sync, nextTick, promise, timeout
```

6 Streams & Buffers

Buffer

- Fixed-size chunk of memory for **raw binary data**, stored outside the V8 heap. Global object, subclass of `Uint8Array`.
- `Buffer.from()`, `Buffer.alloc(n)` (zero-filled, safe), `Buffer.allocUnsafe(n)` (faster, may hold old data), `Buffer.concat()`, `toString(encoding)`.

Streams

- Handle data in **chunks**: memory efficient, time efficient, composable. All streams are **EventEmitters**.
- **4 types**: **Readable** (`fs.createReadStream, req`), **Writable** (`fs.createWriteStream, res`), **Duplex** (TCP socket; two independent sides), **Transform** (Duplex that modifies data, e.g. `zlib.createGzip`).
- **Events**: `data`, `end`, `error`, `close`, `finish`, `drain`.
- **Piping**: `readable.pipe(writable)`. Prefer `stream.pipeline()` for proper error handling and cleanup.
- **Backpressure**: source is faster than destination; `write()` returns `false`, wait for `drain`. `pipe` and `pipeline` handle it automatically.
- **highWaterMark**: internal buffer size (default 16 KB; 64 KB for `fs` read streams).
- Modes: flowing and paused.

7 NPM & Package Management

- **NPM** = online registry + CLI tool + dependency manager. Alternatives: Yarn, pnpm.
- **package.json**: metadata, **scripts**, **dependencies**, **devDependencies**, **main**, **engines**, **type**.
- **package-lock.json**: exact versions of the full dependency tree. **Commit it**. Never edit manually.
- **node_modules**: installed packages. **Never commit**; add to **.gitignore**.
- **dependencies** = needed in production; **devDependencies** = development only (nodemon, jest). Also **peerDependencies**, **optionalDependencies**.
- **SemVer**: MAJOR.MINOR.PATCH = breaking change . new feature . bug fix.
- **~4.18.2** allows minor and patch updates; **^4.18.2** allows only patch updates.
- **Global** (-g) for CLI tools; **local** (default) for project libraries.
- **npm install** (flexible) vs **npm ci** (exact from lock file, deletes **node_modules**, best for CI/CD).
- **npx** runs a package without installing globally.
- Useful commands: **npm init -y**, **npm i**, **npm i pkg -D**, **npm uninstall**, **npm update**, **npm outdated**, **npm audit**, **npm audit fix**, **npm run**.
- **npm start** and **npm test** do not need **run**.

8 Child Processes, Cluster & Worker Threads

- Problem: Node uses one core, and CPU-heavy tasks block the event loop.
- **Process** has its own memory; **thread** shares memory inside a process.

	Child Process	Cluster	Worker Threads
Creates	New process	Copies of your server (processes)	Threads in same process
Purpose	Run other programs/scripts	Use all CPU cores, more requests	Heavy CPU-bound JS
Memory	Separate	Separate (use Redis to share)	Can share (SharedArrayBuffer)
Example	Python, ffmpeg, git	8 Express copies on 8 cores	Image resize, hashing

- **Child process methods**: **exec** (shell, buffered, small output), **spawn** (stream, large output), **execFile** (no shell, safer), **fork** (Node scripts + IPC messaging).
- Never pass user input to **exec()** (**command injection**).
- **Cluster**: one primary + many workers on the same port, round-robin. Built on **fork()**. Improves **throughput**, not single request speed. Restart dead workers on **exit** event.
- **PM2**: production process manager (**pm2 start app.js -i max**) with auto-restart and monitoring.

9 Error Handling

- **Built-in types**: **Error**, **TypeError**, **ReferenceError**, **SyntaxError**, **RangeError**.
- **Operational errors**: expected, external (DB down, file missing, bad input) → **handle** them.
- **Programmer errors**: bugs (**undefined.name**) → **fix** them; log and restart.
- **Ways to handle**: **try/catch/finally**, error-first callbacks, **.catch()**, **error** event listeners.
- **try/catch** does **not** catch errors in callbacks or un-awaited promises.
- **Custom errors**: extend **Error**, add **statusCode**, **isOperational**, use **Error.captureStackTrace**. Always throw **Error** objects, never strings.
- **Global handlers**: **uncaughtException**, **unhandledRejection**. Use to log, shut down gracefully, restart. Do not keep running.
- **Graceful shutdown**: stop new requests, finish current ones, close DB, exit.
- **error** event with no listener crashes the process.
- Express uses a 4-argument error middleware: (**err**, **req**, **res**, **next**).
- Best practices: do not swallow errors, log properly, do not leak stack traces to users, validate input early.

10 Security & Performance Basics

Environment Variables

- Store secrets/config outside code. Access via **process.env**. Values are always **strings**.
- **dotenv**: loads **.env** into **process.env**. Add **.env** to **.gitignore**, commit **.env.example**.
- Use **NODE_ENV** (development/production).

Security

- **Helmet** (secure HTTP headers), **rate limiting** (`express-rate-limit`), input validation (Joi, zod, `express-validator`), **bcrypt** for password hashing, HTTPS, proper CORS, `npm audit`, body size limit, least privilege.

Attack	What it is	Prevention
SQL/NoSQL Injection	Malicious input alters DB query	Validation, ORM/ODM, sanitize
XSS	Injected script runs in other users' browsers	Escape output, Helmet, CSP
CSRF	Tricks logged-in user's browser into requests	CSRF tokens, SameSite cookies
Brute-force	Many password guesses	Rate limiting, lockout
Command Injection	User input run as a command	Never pass input to <code>exec</code>
DoS/DDoS	Flooding server	Rate limit, load balancer, CDN

Performance

- **Never block the event loop:** avoid sync methods in handlers, offload CPU work to worker threads.
- Use async code, `Promise.all`, streams for big data.
- **Caching:** in-memory (not shared across workers) or **Redis** (shared). Also HTTP caching and CDN.
- **Compression** (`compression` middleware, `gzip`).
- DB: **indexes**, **pagination**, select needed fields, **connection pooling**.
- Scale with clustering/PM2, load balancer (Nginx), horizontal scaling.
- Production: `NODE_ENV=production`, logger (Winston, Pino), monitoring.

Garbage Collection & Memory Leaks

- V8 frees unreachable memory automatically using **Mark and Sweep**. Young generation (frequent) and old generation (less frequent).
- **Memory leak:** references are kept to data no longer needed. Causes: growing globals, unbounded caches, uncleared `setInterval`, unremoved listeners, closures, unclosed resources.
- Detect: `process.memoryUsage()`, `node -inspect` with heap snapshots.
- Prevent: clear timers, remove listeners, use LRU cache or Redis.